[부록] JSI 의 블로킹 방지 방법

1. 비동기 유지 (Promise / Callback)

핵심 이론: Event Loop + Microtask Queue

```
Call Stack
   → Web API (Native 처리)
   → Callback / Microtask Queue
   → Call Stack (재진입)
```

JSI 동기 호출이라도 await 로 감싸면 JS Thread 는 즉시 제어권을 반환합니다.

Promise 는 Microtask Queue(빠르게 처리되는 작업 큐)에 등록되어 현재 Call Stack 이 비워진 직후 실행하므로
→ setTimeout 보다 우선순위 높음

확장 개념: Promise 병렬 패턴

```
// 여러 네이티브 호출 병렬 실행
const [a, b] = await Promise.all([
  nativeModule.taskA(),
  nativeModule.taskB(),
]);

// 실패해도 전체를 중단하지 않을 때
const results = await Promise.allSettled([...]);
```

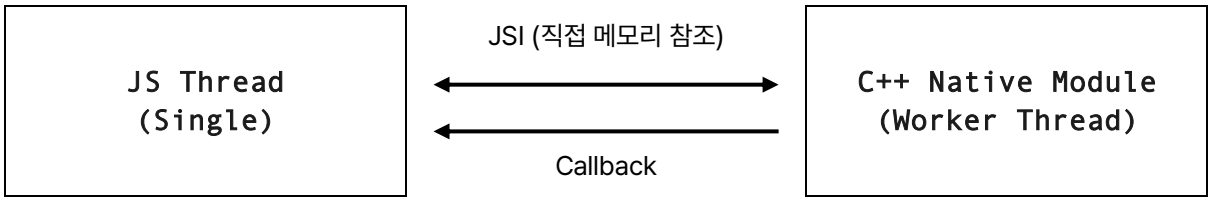
왜 '가벼운 동기 호출 외엔 안전'인가?

JSI 동기 호출 자체는 JS Thread 를 블로킹하므로, 무거운 연산을 동기로 호출하면 16ms 프레임 예산을 초과
→ Frame Drop 발생

※ 60fps 기준 1 프레임 = 16.6ms, 120fps 기준 = 8.3ms

2. 스레드 위임 (C++ Worker Thread)

핵심 이론: React Native 스레드 모델



Old Architecture 의 Bridge 와 달리 JSI 는 직접 메모리 참조 → 직렬화(serialization) 비용 없음

Worker Thread 는 OS 스케줄러가 관리 → JS Thread 와 독립적으로 병렬 실행

확장 개념: Thread Safety

C++ 레벨에서 경쟁 조건(Race Condition) 방지를 위해 다음을 사용함:

- `std::mutex` — 임계 영역(Critical Section) 상호 배제
- `std::atomic` — lock-free 단순 타입 공유

New Architecture 에서의 변화 (TurboModules)

TurboModules 가 이 패턴을 공식화 → 필요할 때만 모듈 로딩(Lazy Loading) + Worker Thread 위임

```
// TurboModule 인터페이스 예시
TurboModuleRegistry.getEnforcing<Spec>('HeavyTask')
  .processAsync(data)        // C++ Worker 에서 실행
  .then(result => { ... });  // JS Thread 에서 콜백
```

3. Worklets (Reanimated 3)

핵심 이론: UI Thread 직접 실행

```
일반 애니메이션:    JS Thread → Bridge → UI Thread  (2 hop, 지연 발생)
Worklet 애니메이션:                      UI Thread  (0 hop, 즉각 반응)
```

'worklet' 지시어 → Reanimated 가 해당 함수를 별도 JS 런타임(JSI 기반)으로 직렬화

UI Thread 자체에 JS 인터프리터를 올려서 실행 → JS Thread 가 frozen 되어도 애니메이션 지속

확장 개념: Shared Values (useSharedValue)

```
// UI-JS Thread 간 공유 상태
const offset = useSharedValue(0);

// UI Thread 에서 직접 실행 (JS Thread 불필요)
const animatedStyle = useAnimatedStyle(() => {
  'worklet';
  return { transform: [{ translateX: offset.value }] };
});
```

왜 판도를 바꿨다고 평가받을까요?

60fps/120fps 보장 조건: 각 프레임마다 16.6ms/8.3ms 내 렌더 완료 필요

JS Thread 의존을 제거함으로써 GC 일시정지나 JS 연산 폭주에도 애니메이션이 끊기지 않음

- Gesture Handler 와 결합 시 터치 → 애니메이션 전체 흐름이 UI Thread 에서 완결
- React Native Skia 도 동일한 Worklet 방식으로 UI Thread 직접 렌더링

세 방법의 이론적 비교 요약

구분	비동기 유지	스레드 위임	Worklets
실행 위치	JS Thread	C++ Worker	UI Thread
JS Thread 점유	최소화	없음	없음
사용 난이도	낮음	높음 (C++)	중간
적합한 작업	I/O, API 호출	무거운 연산	애니메이션, 제스처
프레임 드롭 방지	간접적	완전 분리	완전 분리

그래서 핵심은?

JS Thread 는 싱글 스레드이므로 “Blocking 발생 = Frame Drop”입니다.
따라서 세 방법은 'JS Thread 에서 얼마나 멀어지는가'라는 스펙트럼으로 이해할 수 있습니다.

비동기 유지 (JS Thread 내, 최소화) → 스레드 위임 (완전 분리, C++) → Worklets (완전 분리, UI Thread)